

I PROBLEMI CLASSICI DELLA CONCORRENZA E COME RISOLVERLI



SOMMARIO

CONCORRENZA.....	3
COS'È LA CONCORRENZA.....	3
VANTAGGI.....	3
SVANTAGGI.....	3
PROBLEMI CLASSICI DELLA CONCORRENZA.....	4
PRODUTTORE E CONSUMATORE.....	4
POSSIBILI PROBLEMI.....	4
CODICE RISOLUTIVO.....	4
FILOSOFI A CENA.....	5
POSSIBILI PROBLEMI.....	5
COME RISOLVERE.....	5
LETTORE E SCRITTORE.....	6
POSSIBILI PROBLEMI.....	6
COME RISOLVERE.....	6
BARBIERE SONNOLENTO.....	7
POSSIBILI PROBLEMI.....	7
COME RISOLVERE.....	7

CONCORRENZA

COS'È LA CONCORRENZA

In informatica, la concorrenza tra processi è la capacità di un sistema operativo di gestire più processi (thread) che risultano attivi nello stesso lasso di tempo. Fondamentalmente si verifica quando l'esecuzione di più compiti si sovrappone, portando alla «competizione» per l'accesso limitato delle risorse come CPU, RAM e I/O.

VANTAGGI

- **MAGGIORE EFFICIENZA:** La CPU non resta mai inattiva: mentre un processo attende un'operazione I/O, un altro può essere eseguito.
- **MAGGIORE REATTIVITÀ:** Il sistema può gestire molte operazioni in parallelo (applicazioni multitasking).
- **SCALABILITÀ:** Su sistemi multicore, più processi possono essere eseguiti simultaneamente.
- **MODULARITÀ:** Suddividere un programma in processi/thread semplifica lo sviluppo e l'organizzazione del codice.

SVANTAGGI

- **RACE CONDITION:** Due processi accedono contemporaneamente alla stessa risorsa così portando a risultati imprevedibili.
- **DEADLOCK:** Due o più processi restano bloccati in attesa l'uno dell'altro e così nessuno può procedere.
- **MAGGIORE COMPLESSITÀ DI PROGETTAZIONE:** Programmare sistemi concorrenti è molto più complicato rispetto a programmi sequenziali.

PROBLEMI CLASSICI DELLA CONCORRENZA

PRODUTTORE E CONSUMATORE

Il problema Produttore-Consumatore è un classico scenario di programmazione concorrente dove un thread (il produttore) genera dati/risorse e un altro (il consumatore) li elabora, usando un buffer condiviso (area di memoria limitata). Per evitare che il produttore scriva su buffer pieno o il consumatore legga da buffer vuoto si utilizza la sincronizzazione, tipicamente semafori e `wait()` / `notify()` per coordinare i thread.

POSSIBILI PROBLEMI

- **Race condition:** Produttori e consumatori accedono contemporaneamente al buffer. Senza coordinazione, un produttore potrebbe sovrascrivere dati non ancora consumati o un consumatore potrebbe leggere dati inconsistenti.
- **Mutua esclusione:** l'accesso al buffer deve avvenire uno alla volta, è necessario impedire che più thread modifichino la struttura dati nello stesso momento.
- **Deadlock:** se produttori e consumatori aspettano condizioni che non vengono mai soddisfatte (ad esempio per un uso errato dei meccanismi di attesa), il sistema può bloccarsi.
- **Starvation:** una pessima gestione delle priorità può causare l'attesa indefinita di alcuni produttori o consumatori.

CODICE RISOLUTIVO

- **Sincronizzazione del Buffer (Mutua Esclusione):** È il fattore primario. Poiché il buffer è una risorsa condivisa, l'accesso deve essere atomico. Bisogna garantire che un solo thread alla volta (produttore o consumatore) possa modificare la struttura dati per evitare corruzioni o perdite di informazioni.
- **Gestione del Buffer (Buffer Pieno):** Bisogna risolvere il caso in cui il produttore è più veloce del consumatore. Il sistema deve sospendere il produttore quando il buffer raggiunge la capacità massima, impedendo il tentativo di inserire dati in una memoria già piena.
- **Gestione del Buffer (Buffer Vuoto):** Specularmente, bisogna gestire il caso in cui il consumatore è più veloce. Il consumatore deve essere sospeso se il buffer è vuoto, evitando che tenti di prelevare dati inesistenti o "nulli".
- **Eliminazione dell'Attesa Attiva (Busy Waiting):** È il fattore di efficienza. Invece di far girare i thread in un ciclo continuo che controlla lo stato del buffer (consumando CPU inutilmente), bisogna implementare un meccanismo di sospensione e risveglio (`wait` e `notify`) che metta i thread a riposo finché non possono effettivamente operare.
- **Notifica e Risveglio (Signaling):** Occorre assicurarsi che ogni volta che un produttore inserisce un dato, invii un segnale per svegliare un consumatore in attesa, e che ogni volta che un consumatore preleva un dato, segnali al produttore che si è liberato spazio. Il rischio da risolvere è il "segnale mancato", che porterebbe entrambi i thread in uno stallo permanente.

FILOSOFI A CENA

Il "Problema dei Filosofi a Cena" è un classico esempio di problema di controllo della concorrenza e sincronizzazione dei processi in Java, dove cinque filosofi mangiano ma per mangiare hanno bisogno di due forchette (una per lato), creando il rischio di un deadlock (stallo). Infatti se prendono una forchetta e aspettano l'altra, non potendola mai prendere se il vicino la tiene. Il problema dei 5 filosofi a cena mostra la difficoltà di far condividere risorse limitate a più processi senza causare blocchi, attese infinite o conflitti. È un modello che serve a capire come evitare deadlock, starvation e problemi di sincronizzazione nei sistemi concorrenti.

POSSIBILI PROBLEMI

- **Deadlock:** Situazione in cui tutti i filosofi prendono una forchetta (ad esempio quella di sinistra) e poi restano in attesa della seconda, che però è già occupata da un vicino.
- **Starvation:** Uno o più filosofi non riescono mai a ottenere le due forchette necessarie, perché altri filosofi sono più "fortunati" o vengono serviti più spesso.
- **Livelock:** I filosofi continuano a rilasciare e riprendere le forchette, ma non riescono mai a mangiare.
- **Poor Utilization:** Le strategie di sincronizzazione troppo rigide possono ridurre l'efficienza, ad esempio permettendo di mangiare a pochi filosofi anche se le risorse ci sono.

COME RISOLVERE

- **Prevenzione del Deadlock (Stallo):** È il fattore più noto. Se ogni filosofo prende contemporaneamente la forchetta alla sua sinistra, tutti rimarranno in attesa infinita della forchetta destra. Bisogna "sistemare" la logica di acquisizione (ad esempio rompendo la simmetria o limitando il numero di filosofi al tavolo) per impedire questa configurazione di blocco totale.
- **Prevenzione della Starvation:** Bisogna garantire che ogni filosofo, prima o poi, riesca a mangiare. In alcune soluzioni, un filosofo potrebbe essere "saltato" continuamente dai suoi vicini che si alternano nell'uso delle forchette, morendo di fame virtuale mentre gli altri mangiano. Il sistema deve garantire l'equità di accesso.
- **Mutua Esclusione delle Risorse:** Ogni forchetta è una risorsa critica che può essere utilizzata da un solo filosofo alla volta. Il codice deve garantire che non sia fisicamente possibile per due filosofi adiacenti utilizzare la stessa forchetta nello stesso momento, proteggendo l'integrità del "possesso" della risorsa.
- **Gestione delle atomicità delle azioni:** Spesso è utile risolvere il problema facendo in modo che un filosofo possa prendere le forchette solo se sono entrambe disponibili. Questo richiede che l'operazione di "controllo e acquisizione" avvenga in modo atomico, per evitare che un filosofo prenda la prima, ma la seconda gli venga sottratta un istante prima di afferrarla.
- **Sincronizzazione degli Stati:** Ogni thread-filosofo deve gestire correttamente il passaggio tra gli stati di "pensiero", "fame" e "pasto". La comunicazione dello stato ai vicini o a un coordinatore centrale è essenziale per decidere chi può procedere quando una forchetta viene posata.

LETTORE E SCRITTORE

Il “problema del Lettore/Scrittore” (Reader/Writer) in Java riguarda la gestione della concorrenza per l'accesso a una risorsa condivisa, dove più lettori possono leggere simultaneamente ma uno scrittore blocca tutti gli altri (lettori e scrittori). questo problema viene gestito tramite semafori (come semaphore o synchronized) per garantire che le operazioni siano sicure e ordinate (es. FIFO), prevenendo race condition e garantendo l'integrità dei dati in un ambiente multithread.

- lettura = accesso condivisibile
- scrittura = accesso esclusivo

Questo concetto è alla base di tutte le soluzioni al problema e spiega perché servono meccanismi di sincronizzazione diversi rispetto alla semplice mutua esclusione.

POSSIBILI PROBLEMI

- **Starvation** la starvation si verifica quando un tipo di processo, tipicamente lo scrittore, rimane indefinitamente in attesa perché l'altro tipo ha sempre la precedenza.
- **Deadlock** Tutti i processi aspettano che qualcun altro liberi la risorsa, ma nessuno riesce ad entrare.
- **Race condition:** lettori e scrittori accedono contemporaneamente al buffer. Senza coordinazione, uno scrittore potrebbe sovrascrivere dati non ancora consumati o un lettore potrebbe leggere dati inconsistenti.

COME RISOLVERE

- **Mutua Esclusione (Integrità del Dato):** È il fattore fondamentale per evitare la corruzione della risorsa. Bisogna garantire che l'accesso in scrittura sia strettamente esclusivo (uno scrittore alla volta e nessun lettore presente), mentre l'accesso in lettura può essere condiviso tra più thread simultaneamente.
- **Prevenzione della Starvation (Inanizione):** Bisogna decidere una politica di precedenza. Senza una gestione oculata, uno scrittore potrebbe non riuscire mai a scrivere perché nuovi lettori continuano ad arrivare "saltando la fila" (starvation dello scrittore), o viceversa. La soluzione tipica consiste nell'implementare una politica di **Fairness** (equità) che rispetti l'ordine di arrivo delle richieste.
- **Gestione del Throughput (Efficienza):** Un fattore chiave è massimizzare il parallelismo. Se il sistema fosse risolto con un semplice synchronized (che blocca tutto indistintamente), si perderebbe il vantaggio di permettere a più lettori di lavorare insieme. Bisogna quindi "sistemare" il codice affinché i lettori non si blocchino a vicenda, ma blocchino solo lo scrittore.
- **Sincronizzazione dei Segnali (Condition Variables):** Occorre un meccanismo per cui, quando uno scrittore finisce, deve "svegliare" sia eventuali scrittori che lettori in attesa, e quando l'ultimo lettore finisce, deve segnalare specificamente allo scrittore in attesa che la risorsa è finalmente libera.
- **Prevenzione del Deadlock:** In implementazioni complesse (dove magari un lettore deve diventare scrittore o viceversa), bisogna assicurarsi che i thread non rimangano bloccati aspettando l'uno il rilascio del lock dell'altro, garantendo che ogni acquisizione segua un percorso logico lineare.

BARBIERE SONNOLENTO

Il "Barbiere SonnoLENTO" è un classico problema di sincronizzazione in java, che simula un barbiere che dorme se non ci sono clienti e si sveglia per servirne uno, mentre altri clienti si mettono in coda. Se la sala d'attesa si riempie, i nuovi clienti se ne vanno, illustrando la gestione di risorse concorrenti (barbiere/sedia) e processi (barbiere/clienti) in sistemi operativi, spesso risolto con semafori.

POSSIBILI PROBLEMI

- **Lost wake-up:** Se il segnale di risveglio non viene gestito correttamente, il barbiere può finire per dormire anche se c'è un cliente in attesa, causando un blocco del servizio.
- **Mancanza di mutua esclusione:** Più clienti potrebbero essere serviti insieme se non gestite bene le risorse.
- **Gestione errata delle risorse:** Le sedie nella sala d'attesa sono in numero finito. Il sistema deve garantire che il numero di clienti in attesa non superi quello delle sedie disponibili e che i clienti in eccesso vengano correttamente rifiutati.
- **Deadlock o attese infinite:** Barbiere e clienti restano bloccati ad aspettarsi a vicenda. Questo può succedere se i meccanismi di attesa e segnalazione sono progettati in modo errato.

COME RISOLVERE

- **Gestione degli Stati (Dormiveglia):** Bisogna risolvere il meccanismo di "sveglia". Il barbiere deve passare dallo stato di attesa passiva (sonno) a quello attivo solo quando il primo cliente arriva, evitando che il barbiere consumi risorse CPU mentre aspetta o che il cliente rimanga in attesa se il barbiere è già disponibile.
- **Mutua Esclusione sulle Risorse Limitate:** Il numero di sedie libere nella sala d'aspetto è una risorsa condivisa. È fondamentale risolvere il rischio di Race Condition: senza un mutex, due clienti che arrivano contemporaneamente potrebbero vedere lo stesso posto libero e cercare di occuparli entrambi, o corrompere il contatore delle sedie.
- **Protocollo di Signaling (Sincronizzazione):** Occorre un sistema di segnali bidirezionali. Il cliente deve segnalare al barbiere che è pronto (invocando il servizio), e il barbiere deve segnalare al cliente quando il taglio è terminato, in modo che il cliente non lasci la sedia prematuramente e il barbiere possa passare al prossimo compito.
- **Gestione della Capacità (Bouncing dei Clienti):** Bisogna gestire correttamente la condizione di "negoziO pieno". Il sistema deve decidere istantaneamente, nel momento in cui un thread-cliente viene creato, se questo può entrare in coda o se deve terminare immediatamente (andarsene) senza restare bloccato in attesa di una sedia che non c'è.
- **Prevenzione dell'Inconsistenza (Lost Wake-up):** È un fattore tecnico cruciale: bisogna assicurarsi che il segnale di "cliente arrivato" non vada perduto se il barbiere sta proprio in quel momento passando dal taglio dei capelli al sonno. L'uso dei semafori risolve questo problema mantenendo memoria dei segnali inviati.